AskMyPDF

Retrieval-Augmented Chat for Documents

Project brief · Author: Moaz · October 2026 · Development and hosting cost: $0

1. Project Overview

AskMyPDF is a web application that lets users upload PDF documents, ask questions in plain language, and receive answers drawn only from those documents, with the file name and page number of every source. It uses Retrieval-Augmented Generation (RAG): the system first retrieves the most relevant passages from the PDFs, then a large language model (LLM) writes the answer using only those passages. Every tool is free and open source; the LLM runs offline on the user's laptop (Ollama) or online through Groq's free tier, and the demo is hosted free on Streamlit Community Cloud.

2. Problem Statement

Students and professionals spend significant time searching long PDFs such as reports, research papers, manuals and policies. General chatbots answer from training data, may hallucinate, and cannot cite where an answer came from. AskMyPDF answers from the user's own documents and shows the exact page, so every answer can be verified.

3. Objectives

- Build a working RAG pipeline: PDF loading, chunking, embedding, vector search and grounded generation
- Provide page-level citations for every answer
- Refuse to answer when the information is not in the documents
- Support fully offline use (Ollama) and fast online use (Groq), selectable in the interface
- Evaluate the system with measurable metrics and tune it from the results
- Deploy a public demo at zero cost and publish a portfolio-ready GitHub repository

4. Scope

In scope	Out of scope (version 1)
Text-based PDFs, multiple files per session	Scanned/image-only PDFs (OCR is a stretch goal)
English questions and documents	User accounts and login
Single-user local app and public demo	Multi-user persistent cloud storage
Streamlit interface	Docker, mobile app

5. Requirements

5.1 Functional requirements

ID	Requirement	Priority
FR-1	Upload one or more text-based PDFs from the sidebar	Must
FR-2	Extract text page by page, keeping file name and page number	Must
FR-3	Split text into overlapping chunks	Must
FR-4	Embed chunks and store them in a persistent vector database	Must
FR-5	Accept natural-language questions through a chat interface	Must

ID	Requirement	Priority
FR-6	Retrieve the top-k most similar chunks for each question	Must
FR-7	Generate answers using only retrieved content, with page citations	Must
FR-8	Reply "I couldn't find that in the uploaded documents" when the answer is absent	Must
FR-9	Stream the answer as it is generated	Should
FR-10	Show the retrieved source chunks on request	Should
FR-11	Switch the LLM between Ollama (offline) and Groq (online)	Should
FR-12	Keep processed documents across restarts; allow clearing the database	Should
FR-13	Keep chat history visible during the session	Should

5.2 Non-functional requirements

Area	Requirement
Cost	$0 for development, models and hosting; no paid APIs and no credit card
Performance	Answer within about 10 s on a laptop (Ollama 3B) and about 3 s on Groq; a 50-page PDF processed in under 1 minute
Accuracy	Retrieval hit rate of 80% or more and correct refusals of 80% or more on a 20-question test set
Reliability	Clear error messages for a stopped Ollama server, missing API key or Groq rate limit
Security	API key stored in .env locally and in Streamlit secrets online; never committed to GitHub
Privacy	In offline mode, documents never leave the user's machine
Usability	Single-page interface usable without technical knowledge
Maintainability	RAG logic separated from the UI in its own package; settings in one config file

5.3 Hardware and software requirements

Item	Minimum	Recommended
RAM	4 GB (LLM on Groq)	8 GB or more (LLM on Ollama)
Storage	3 GB free	5 GB free
CPU / GPU	Any modern CPU	GPU optional; speeds up Ollama
Operating system	Windows 10, macOS or Linux	Same
Software	Python 3.11, Git, Ollama, web browser	VS Code as editor
Internet	Setup, model downloads, Groq mode, deployment	Not needed for offline use
Accounts (free)	GitHub, Groq, Streamlit Community Cloud	Same

6. Tech Stack (all free)

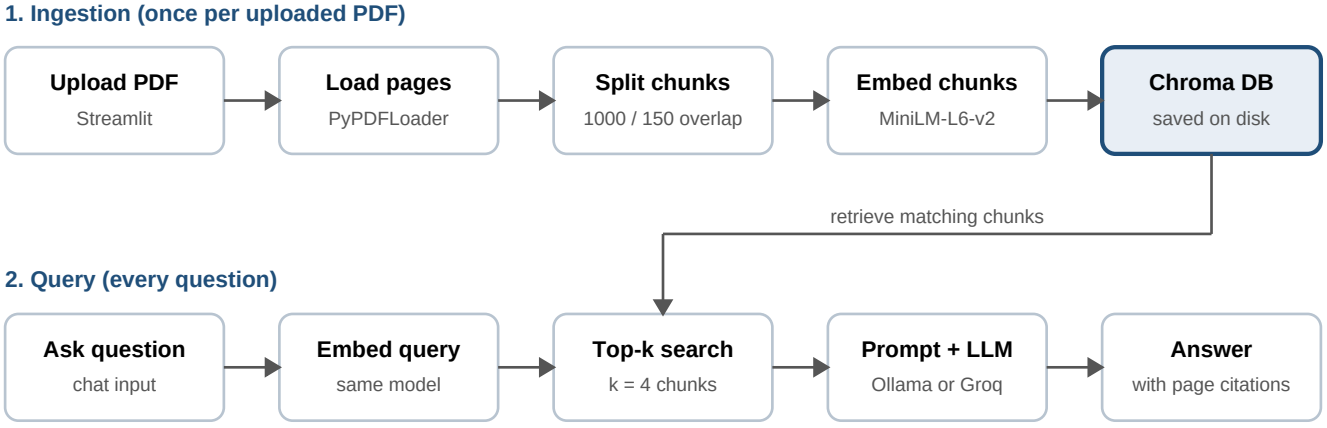
Layer	Tool / package	Purpose	Cost
Language	Python 3.11	Core development language	Free
RAG framework	langchain, langchain-community, langchain-core	Loaders, splitters, prompts, chains	Open source
Text splitting	langchain-text-splitters (RecursiveCharacterTextSplitter)	Split pages into overlapping chunks	Open source

Layer	Tool / package	Purpose	Cost
PDF parsing	pypdf via PyPDFLoader	Extract text with page numbers	Open source
Embeddings	sentence-transformers/all-MiniLM-L6-v2 via langchain-huggingface	Text to 384-dimension vectors on CPU (~90 MB model)	Open source
Vector database	ChromaDB via langchain-chroma	Local persistent storage and similarity search	Open source
LLM (offline)	Ollama + Llama 3.2 3B (alt: Qwen 2.5 3B / 1.5B) via langchain-ollama	Local answer generation, no internet	Free
LLM (online)	Groq API (llama-3.1-8b-instant) via langchain-groq	Fast hosted generation	Free tier, rate-limited
User interface	Streamlit	Upload, chat, streaming, citations	Open source
Configuration	python-dotenv	Load API key from .env	Open source
Evaluation	Custom Python script; RAGAS (optional)	Hit rate, refusals, faithfulness	Open source
Version control	Git + GitHub	Source code and portfolio	Free
Hosting	Streamlit Community Cloud	Public demo deployed from GitHub	Free
Editor	VS Code	Development environment	Free

Groq's model list changes over time; confirm a current small model at console.groq.com/docs/models before starting. Free-tier limits apply per organization and are listed at console.groq.com/docs/rate-limits.

7. System Architecture

The system has two pipelines that share one vector database. Ingestion runs once per uploaded PDF; the query pipeline runs for every question.



8. Methodology

AskMyPDF follows the RAG approach and an iterative development method: build a working baseline, measure it on a fixed test set, then change one parameter at a time and keep only changes that improve the numbers.

Stage	Process	Key settings
1. Document loading	PyPDFLoader reads each page into a document with file name and page number metadata	Page-level metadata

Stage	Process	Key settings
2. Chunking	RecursiveCharacterTextSplitter splits on paragraphs, then lines, sentences and words, so chunks end at natural breaks	1000 characters, 150 overlap
3. Embedding	Each chunk is converted to a 384-dimension vector with all-MiniLM-L6-v2	CPU, model cached in memory
4. Storage	Vectors, text and metadata stored in ChromaDB on disk with stable IDs (file name + index) to avoid duplicates	Persistent folder
5. Retrieval	The question is embedded with the same model; similarity search returns the closest chunks	Top-k = 4
6. Augmentation	Retrieved chunks, each labelled [file, page], are inserted into a prompt that restricts the LLM to that context	Grounding prompt
7. Generation	The LLM writes the answer with citations, or the fixed refusal sentence	Temperature 0
8. Evaluation	A 20-question test set measures retrieval hit rate and correct refusals; chunk size, overlap and top-k are tuned	15 answerable, 5 not

9. Features

9.1 Core (version 1)

- Multi-PDF upload and processing with progress messages
- Chat interface with streamed answers and session history
- Page-level citations under every answer
- Expandable view of the exact chunks used
- Refusal when the answer is not in the documents
- Ollama / Groq model switch
- Persistent storage and a Clear documents button

9.2 Stretch goals (version 2)

- Conversation memory: rewrite follow-up questions into standalone questions
- Better retrieval: MMR search, or reranking with the free BAAI/bge-reranker-base model
- Filter questions to one selected PDF using metadata
- One-click document summary
- OCR for scanned PDFs with Tesseract
- Settings panel to change chunk size, overlap and top-k live
- Urdu support with paraphrase-multilingual-MiniLM-L12-v2 embeddings

10. Project Structure

```
askmypdf/
|-- app.py                # Streamlit user interface
|-- rag/
|   |-- __init__.py
|   |-- config.py         # chunk size, model names, paths
|   |-- ingest.py         # load PDF -> split -> embed -> store
|   |-- retriever.py      # similarity search
|   |-- llm.py            # Ollama or Groq
|   `-- chain.py          # prompt + LLM + citations
|-- eval/
|   |-- __init__.py
|   |-- questions.csv     # test questions and expected pages
|   `-- run_eval.py
|-- data/uploads/         # uploaded PDFs (gitignored)
|-- chroma_db/            # vector store (gitignored)
```

```
|-- .env                  # GROQ_API_KEY (gitignored)
|-- .gitignore
|-- requirements.txt
`-- README.md
```

11. Environment Setup

- **Python 3.11** from python.org (tick "Add Python to PATH" on Windows)
- **Ollama** from ollama.com, then download the model

```
ollama pull llama3.2:3b
ollama run llama3.2:3b "Say hello"      # quick test
```

- **Groq key** (optional, for online mode): sign up at console.groq.com, open API Keys, create a key; no card required
- **Project folder and virtual environment**

```
mkdir askmypdf && cd askmypdf
python -m venv venv
venv\Scripts\activate            # Windows
source venv/bin/activate          # macOS / Linux
```

requirements.txt

```
langchain
langchain-community
langchain-chroma
langchain-huggingface
langchain-ollama
langchain-groq
sentence-transformers
pypdf
streamlit
python-dotenv
```

```
pip install -r requirements.txt
```

.env (add .env, venv/, chroma_db/ and data/uploads/ to .gitignore)

```
GROQ_API_KEY=your_key_here
```

The embedding model (about 90 MB) downloads once on first run; afterwards everything except Groq mode works offline.

12. Development Phases

Phase	Tasks	Deliverable	Done when	Time
1. Setup	Install Python, Ollama, packages; Groq key; GitHub repo	Working environment	ollama run answers; imports work	Day 1
2. Ingestion	config.py and ingest.py: loading, chunking, embedding, storage	Searchable vector store	Chunk count prints; chroma_db/ exists	Days 2-3
3. Retrieval	retriever.py; check chunk relevance	Retriever module	Retrieved chunks match test questions	Day 4
4. Generation	llm.py and chain.py: LLM, grounding prompt, citations	Cited answers in terminal	Answers cite correct pages; refusals work	Days 5-6
5. Interface	app.py: upload, chat, streaming, sources, model switch	Working web app	Full flow works in the browser	Days 7-8
6. Evaluation	Test set, run_eval.py, tuning experiments	Results table	Hit rate and refusal rate recorded	Day 9

Phase	Tasks	Deliverable	Done when	Time
7. Deployment	README, screenshots, Streamlit Cloud	Public demo link	Link works for others	Day 10
8. Enhancement	Stretch goals, re-evaluate after each	Version 2	Each change improves a metric	Days 11-17

Estimated at 2-3 hours per day: about 10 days for the core app and one more week for enhancements.

13. Testing and Evaluation

13.1 Metrics

Metric	Definition	Target
Retrieval hit rate	Share of answerable questions where the correct page is in the top-k chunks	80% or more
Correct refusal rate	Share of unanswerable questions that receive the refusal sentence	80% or more
Faithfulness (optional, RAGAS)	How far the answer is supported by the retrieved chunks	0.8 or more
Response time	Seconds from question to complete answer	10 s local, 3 s Groq

13.2 Test set (eval/questions.csv)

Twenty questions on one PDF you know well: 15 answerable, with the page of the answer, and 5 not in the document.

```
question,expected_page,answerable
What sample size did the study use?,3,yes
Which method performed best?,7,yes
Who won the 2022 World Cup?,,no
```

Run with `python -m eval.run_eval` (script in Appendix A.7).

13.3 Tuning experiments

Chunk size	Overlap	Top-k	Hit rate	Correct refusals
500	50	4		
1000	150	4		
1500	200	6		

Delete chroma_db/ and re-ingest before each experiment so old chunks do not mix in.

13.4 Manual test checklist

- A question answered on the first page and one on the last page both work
- With two PDFs uploaded, citations name the correct file
- An unrelated question receives the refusal sentence
- Clear documents empties the database
- Processed PDFs are still searchable after restarting the app (local)
- Stopping Ollama or removing the Groq key shows a clear error, not a crash

14. Deployment (Streamlit Community Cloud, free)

Ollama cannot run on Streamlit Community Cloud, so the public demo uses Groq; the local version keeps both models.

Step	Action
1	Push the project to a public GitHub repository; confirm .gitignore excludes .env, venv/, chroma_db/ and data/uploads/
2	Sign in at share.streamlit.io with GitHub, click Create app, select the repository and set the main file to app.py
3	Open Advanced settings and select Python 3.11
4	Add secrets (top-level secrets are also available as environment variables): GROQ_API_KEY = "..." and ON_CLOUD = "1"
5	Deploy; the first start takes a few minutes to install packages and download the embedding model
6	Test the public link from another device and add it to the README

Hosting limits: cloud storage resets when the app restarts, so visitors upload their own PDFs each session; apps sleep after inactivity and wake when opened; Groq free limits are shared by all demo visitors (HTTP 429 when reached). If the key is ever committed by mistake, delete it in the Groq console and create a new one.

15. Risks and Solutions

Problem	Cause	Solution
Empty or garbled answers	Scanned PDF with no text layer	Use text PDFs; add Tesseract OCR later
Connection refused on port 11434	Ollama not running	Start the Ollama app or run ollama serve
Laptop slow or freezing	3B model too heavy for available RAM	Use qwen2.5:1.5b or switch to Groq
HTTP 429 from Groq	Free rate limit reached	Wait a minute; use Ollama meanwhile
Groq model not found	Model retired by Groq	Choose a current model from the Groq models page
Stale or duplicate answers	Old chunks still stored	Clear documents or delete chroma_db/
Correct page never retrieved	Chunk size or top-k unsuitable	Run tuning experiments; raise top-k to 6
Model ignores the context rule	Small models drift	Keep temperature 0; repeat the rule in the user message
Embedding model slow to load	Model reloaded on every call	Cache it with lru_cache (already in ingest.py)

16. Expected Outcomes

- A deployed PDF question-answering app with page-level citations and refusal behaviour
- Measured results for retrieval hit rate and refusal accuracy, with a tuning table
- A clean GitHub repository with README, screenshots, architecture diagram and demo link
- Hands-on skills: document processing, embeddings, vector databases, prompt design, LLM integration, evaluation and deployment
- Total cost: $0

16.1 Resume bullets (fill in measured numbers)

- Built AskMyPDF, a retrieval-augmented PDF question-answering app using LangChain, ChromaDB and Streamlit, with page-level citations for every answer
- Supported fully offline inference with Llama 3.2 via Ollama and a Groq cloud option; deployed at zero cost
- Evaluated on a 20-question test set: __% retrieval hit rate and __% correct refusals after tuning chunk size and top-k